2.1 Thinking in Classes and Objects

Chapter 17 introduces a new way to organize familiar PHP ideas. A property is a variable attached to an object, and a method is a function attached to an object. OOP groups related data and behavior into classes so that the code resembles the problem it solves.

Classes, objects, and messages

A **class** is a reusable description or blueprint. An **object** is one particular instance created from that class. If `Person` is a class, `$nora` and `$sam` can be two objects with different property values but the same available methods.

```php
<?php

declare(strict_types=1);

$nora = new Person();
$sam = new Person();

$nora->setDisplayName('Nora');
$sam->setDisplayName('Sam');

echo $nora->getDisplayName();
```

The expression `$nora->getDisplayName()` sends a **message** to the object by invoking one of its methods. PHP uses `->` to access an object's members. Do not confuse it with `=>`, which separates keys and values in arrays.

Model responsibilities, not just nouns

Finding nouns in a problem description is a useful start, but a good class also has a clear responsibility:

Class	Knows (properties)	Does (methods)
`Person`	display name, email	validate profile changes, create a label
`Course`	title, instructor, registrations	enroll, format for display
`Registration`	student, course, registration time	drop, format for display

Avoid making one giant class responsible for everything. Ask, “Which object has the information needed to do this job?” Put the method there.

Relationships between objects

Properties can contain objects as well as strings and numbers. A `Registration` can hold a reference to the `Person` who is registered:

```php
class Person
{
    public string $displayName;
}

class Course
{
    public Person $instructor;
    public string $title;
    public array $registrations = [];
}

class Registration
{
    public Person $student;
    public Course $course;
    public string $registrationTime;
}

$instructor = new Person();
$instructor->displayName = 'Nora';

$student = new Person();
$student->displayName = 'Sam';

$course = new Course();
$course->instructor = $instructor;
$course->title = 'Introduction to PHP';

$registration = new Registration();
$registration->student = $student;
$registration->course = $course;
$registration->registrationTime = '2024-06-01 10:00:00';
$course->registrations[] = $registration;

echo $registration->student->displayName;
```

This is an object relationship: the registration **has a** student. The type declaration `Person $student` prevents unrelated values from being assigned.

Encapsulation and information hiding

Encapsulation groups an object's state and behavior inside its class. **Information hiding** limits which code may access that state. If any code could assign `-10` to an account balance or an empty string to a required name, the object could become invalid.

A class therefore exposes a small public interface while keeping implementation details private. The rest of the application should care about what the object promises to do, not how it stores every value.

A preview of inheritance and overriding

Chapter 17 also introduces an **is-a** relationship. A `Student` is a specialized `Person`, so it might inherit shared properties and methods:

```
class Person
{
    public function roleLabel(): string
    {
        return 'Community member';
    }
}

class Student extends Person
{
    public function roleLabel(): string
    {
        return 'Student';
    }
}

class Teacher extends Person
{
    public function roleLabel(): string
    {
        return 'Teacher';
    }
}
```

`Student` and `Teacher` **override** `roleLabel()` by supplying more specific behavior. For now, recognize superclass, subclass, inheritance, and overriding. We will implement inheritance in depth with Chapter 19.

Object-oriented flow of control

OOP does not eliminate sequential execution. A file such as `public/index.php` remains an entry point; it loads classes, creates objects, and calls methods. The following example is a typical entry point for a web application:

```
<?php

declare(strict_types=1);

require_once __DIR__ . '/../src/WebApplication.php';

$app = new WebApplication();
$app->run();
```

Control then moves into `run()` and into other objects that it creates or calls. The entry point answers “where does execution begin?” while the class design answers “which object is responsible for each job?”